## Using a Gemfile

Now that we have a Gemfile, there are three important things that we are most likely to do with it:

- Install the listed gems
- Run a program that is limited to those gems
- Update the gems when new versions are released

### Bundle Install

To install a set of gems, you use the command `bundle install`, which performs a `gem install` on all the gems listed in the Gemfile. If you run the command, you'll see something like this (the exact output depends on what's already installed):

```
$ bundle install
bundle install
Fetching gem metadata from https://rubygems.org/.........
Fetching ast 2.4.2
Installing ast 2.4.2
... MORE GEMS ...
Bundle complete! 6 Gemfile dependencies, 40 gems now installed.
Use `bundle info [gemname]` to see where a bundled gem is installed.
```

If you compare that output to the Gemfile for this book listed earlier, you'll see that the first gem that bundler installs is `ast`, which is interesting, because our Gemfile does not reference `ast`.

What Bundler is doing is installing not just the gems listed in our Gemfile, but also their dependencies and their transitive dependencies all the way down. Bundler downloads those files to exactly the same place that regular `gem install` would, meaning it depends on your operating system and Ruby version manager. Unless otherwise specified, Bundler installs the most recent version of the gem.

When you install your Gemfile with Bundler, Bundler creates a new file, `Gemfile.lock`. The lock file is important because that is where Bundler stores the actual versions of the gems that have been installed.

```
GEM
  remote: https://rubygems.org/
  specs:
    ast (2.4.2)
    backport (1.2.0)
    benchmark (0.2.0)
    concurrent-ruby (1.1.10)
    debug (1.5.0)
      irb (>= 1.3.6)
      reline (>= 0.2.7)

    ... THERE'S MORE, LOCK FILES CAN GET LONG ...

PLATFORMS
  arm64-darwin-21

DEPENDENCIES
  debug
  i18n
```